

Assignment 2: Coding Basics

Yitong Eric Su

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1. Generate a squence from one to 55, increasing by fives. The sequence is named 'seq_by5'  
seq_by5 <- seq(from = 1, to = 55, by = 5)
```

```
#2.  
mean_seq <- mean(seq_by5) #compute the mean of this sequence=26  
median_seq <- median(seq_by5) #compute the median of this sequence=26
```

```
#3. Use logical comparison to determine if the mean is greater than the median  
mean_seq > median_seq #Returns a logical value=FALSE, since both values are the same.
```

```
## [1] FALSE
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.

7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
#5. Create the vectors
student_names <- c("Eric", "Robert", "Michelle", "Hannah") #Character vector
test_scores <- c(98, 93, 88, 95) #Numeric vector
scholarship <- c(TRUE, FALSE, FALSE, TRUE) #Logical vector

#7. Combine the vectors into a data frame
student_data <- data.frame(student_names, test_scores, scholarship)

#8. Label the columns with informative titles
names(student_data) <- c("Names", "Scores", "Scholarship")
```

9. QUESTION: How is this data frame different from a matrix?

Answer: Though matrixes and data frames are both two-dimensional data structures, data frames can contain different types of data (e.g., numeric, character, and logical vectors), while matrixes are simply two-dimensional vectors with the same data type.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word “Pass”; otherwise print the word “Fail”.
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else
check_pass <- function(x){
  if (x>50) {
    print("Pass")
  } else {
    print("Fail")
  }
}

#11. Create a function using ifelse()
check_pass_ifelse <- function(x) {
  ifelse (x>50, "Pass", "Fail")
}

#12a. Run the first function with the value 52.5
check_pass(52.5) #"Pass"
```

```
## [1] "Pass"
```

```
#12b. Run the second function with the value 52.5
check_pass_ifelse(52.5) #"Pass"
```

```
## [1] "Pass"
```

```
#13a. Run the first function with the vector of test scores  
##check_pass(test_scores) #Error: the condition has length > 1
```

```
#13b. Run the second function with the vector of test scores  
check_pass_ifelse(test_scores) #"Pass" "Pass" "Pass" "Pass"
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: In Q13 the ‘ifelse’ function worked, because ‘ifelse’ is vectorized, which means it can apply the condition to each element of the given vector. And as a result, it returned a vector of results, in which each result corresponds to an original element. Meanwhile, the ‘if...else’ function is not vectorized, and therefore, it can only take single elements each time. In other words, ‘ifelse’ can be understood as a vectorized version of ‘if...else’.

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)